# CampusConnect - Reporte Oficial de Entrega

Desarrollo Full Stack con Django, Django REST Framework, JWT, Swagger y React

**Repositorio en GitHub: https://github.com/Jony-777/Backend-con-Django-y-JWT-**

| | | | |
|---|---|---|---|
| **Proyecto:** | CampusConnect (Sistema Escolar) | **Versión API:** | v1 (RESTful) |
| **Backend:** | Django 5.1 + DRF + SimpleJWT | **Frontend:** | React 18 + Vite (JavaScript) |
| **Base de Datos:** | SQLite (excluida de Git) | **Entorno:** | venv (excluido de Git) |
| **Documentación:** | Swagger UI (/swagger/) y ReDoc | **Seguridad:** | Corrección de contraseñas y hashes |

## 1. Captura de Swagger UI con los Endpoints Documentados

La API se encuentra documentada interactivamente en la ruta /swagger/ mediante **drf-yasg**. Muestra todos los endpoints organizados en categorías, con el título **CampusConnect API** y soporte para autenticación Bearer JWT:




## 2. Captura de la Obtención del Token JWT (/auth/token/)

El endpoint POST /api/v1/auth/token/ recibe el correo electrónico y la contraseña del usuario. Al ser válidas las credenciales, devuelve un par de tokens JWT: **access** (vida útil de 30 minutos) y **refresh** (vida útil de 1 día):


Endpoint de Login JWT - POST /api/v1/auth/token/
POST /api/v1/auth/token/ HTTP/1.1
Host: localhost:8000
Content-Type: application/json
{
"email": "admin@campusconnect.edu",
"password": "Admin123456!"
}
HTTP/1.1 200 OK
Content-Type: application/json
{
"refresh": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0b2tlbl90e...",
"access": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0b2tlbl90e..."
}
Token obtenido con éxito. Duración del Access Token: 30 minutos | Refresh Token: 1 día.

## 3. Captura de la Consulta de Datos Personales con el Token (/auth/me/)

El endpoint protegido GET /api/v1/auth/me/ requiere el token JWT en el encabezado Authorization: Bearer <token>. Obtiene automáticamente la información del usuario autenticado desde request.user sin permitir suplantar a otro usuario mediante parámetros. **Garantía de Seguridad:** No devuelve la contraseña ni sus hashes:

Datos Personales - GET /api/v1/auth/me/

```
GET /api/v1/auth/me/ HTTP/1.1
Host: localhost:8000
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0b2tl...

HTTP/1.1 200 OK
Content-Type: application/json
{
  "id": 1,
  "email": "admin@campusconnect.edu",
  "first_name": "Admin",
  "last_name": "CampusConnect",
  "matricula": "ADM-001",
  "is_active": true
}
```

✅ Verificación de seguridad: El campo password y sus hashes NO se exponen en la respuesta.

## 4. Prueba Crítica: Editar un Usuario sin Contraseña NO Cambia su Contraseña

**Corrección del Error de Seguridad del Manual:** El manual de referencia contenía un fallo donde al editar a un usuario sin enviar contraseña, el backend le asignaba la contraseña del administrador solicitante. En CampusConnect se corrigió centralizando la lógica en el método update() de UserSerializer, garantizando que si no se envía el campo password, la contraseña existente permanece **intacta**:

Prueba de Seguridad - Edición sin contraseña (PUT / PATCH)

```
# PASO 1: Comprobación de Login Inicial del alumno con contraseña original
POST /api/v1/auth/token/ -> Credenciales: 'carlos.ramirez@campusconnect.edu' / 'Alumno123456!'
HTTP/1.1 200 OK [✅ Login exitoso inicial]
Hash PBKDF2 inicial en BD: pbkdf2_sha256$870000$OSdtVdy7ffiTCV...

# PASO 2: El Administrador actualiza al alumno mediante PATCH sin enviar campo 'password'
PATCH /api/v1/users/2/ HTTP/1.1 (Solicitado por: admin@campusconnect.edu)
Payload enviado: {"first_name": "Carlos Alberto", "matricula": "MAT-2026-ACTUALIZADA"}
HTTP/1.1 200 OK [✅ Campos actualizados exitosamente]

# PASO 3: Verificación de Login Posterior del alumno con su contraseña original
POST /api/v1/auth/token/ -> Credenciales: 'carlos.ramirez@campusconnect.edu' / 'Alumno123456!'
HTTP/1.1 200 OK [✅ Login exitoso con la contraseña original]

Hash PBKDF2 final en BD:   pbkdf2_sha256$870000$OSdtVdy7ffiTCV...
Hash del Administrador:    pbkdf2_sha256$870000$3OE8sO74He5gGM...

RESULTADO DE SEGURIDAD:
✅✅La contraseña original sigue funcionando?: SÍ (HTTP 200 OK)
✅✅El hash del usuario se mantuvo idéntico?: SÍ (Exactamente igual)
✅✅Se asignó la contraseña del Administrador?: NO (Totalmente aislado)
✅ ERROR DEL MANUAL CORREGIDO EXITOSAMENTE
```

## 5. Verificación de la Suite de Pruebas Automatizadas

Se ejecutaron las 8 pruebas automatizadas en backend/users/tests.py que validan todo el sistema de forma integral:

| Prueba | Descripción | Resultado |
|---|---|---|
| Prueba 1 | Creación segura de usuario y cifrado de contraseña con PBKDF2 | **PASÓ (OK)** |
| Prueba 2 | Login JWT y obtención de tokens access y refresh | **PASÓ (OK)** |
| Prueba 3 | Consulta de datos personales (/auth/me/) sin exponer contraseñas | **PASÓ (OK)** |
| Prueba 4 | Actualización sin contraseña mantiene intacta la clave original (PUT/PATCH) | **PASÓ (OK)** |
| Prueba 5 | Actualización con nueva contraseña actualiza el acceso adecuadamente | **PASÓ (OK)** |
| Prueba 6 | Verificación estricta de que ningún endpoint exponga hashes ni passwords | **PASÓ (OK)** |
| Prueba 7 | Control de permisos: bloqueo a usuarios anónimos y alumnos en CRUD | **PASÓ (OK)** |
| Prueba 8 | Registro público de usuarios (/auth/register/) sin permisos de admin | **PASÓ (OK)** |